COMMUNISERVE SA

Part 3: Final Development, Testing and Submission Portfolio

OPSC6312 | Android Application Development | Revision prepared 4 November 2026

“Discover Local. Connect Community.”

Group members: Precious Moloi | Nyiko Matladi | Luvuyo Yizani | Naledi Paballo Tshelane

1. Project overview

CommuniServe SA is an Android community-services application intended to help South African users discover local services and community events, save useful information for offline access, submit reviews, and contribute new listings for approval. The Android client is written in Kotlin and communicates with a Node.js/Express REST API. Room provides local persistence for saved items and queued actions.

This portfolio describes the implementation represented by the supplied source project. It distinguishes implemented source code from functions that still require configuration or execution evidence. A feature is not treated as passed merely because its source code is present.

2. Implemented solution represented in the source

Android application: Kotlin single-activity application with a shared visual design system, translated text resources, login and registration flows, service and event discovery, listing details, map access, saved/offline content, review workflows, listing submission, profile/settings and approval-related screens.

Backend: Express REST API with registration and login, bcrypt password hashing, JWT sessions, request validation, authentication rate limiting, health endpoint, listing/review/profile/device routes and JSON-file persistence. The JSON data file is excluded from version control. A persistent managed database is recommended for production deployment.

Authentication: email/password authentication is implemented through the API. Google sign-in uses Firebase Authentication on Android and sends a Firebase ID token to the API. The API verifies the token signature, issuer, audience and expiry against Google signing certificates before using the verified email. This requires the project Firebase configuration and FIREBASE_PROJECT_ID on the server.

Offline support: Room entities and DAOs store saved items and queued reviews/listings. The app includes reconnect-triggered synchronisation logic. The exact offline and resynchronisation behaviour still needs to be demonstrated on the target physical device.

Notifications and language: Firebase Cloud Messaging integration and English, Afrikaans and isiZulu string resources are present. Actual push delivery requires Firebase configuration and device permission. Language quality should be proof-read by fluent speakers.

3. User-interface improvements in this revision

The Discover screen now groups related actions under translated section headings to improve scanning and reduce the impression of an undifferentiated list of buttons.

When Google authentication succeeds but the API exchange is unavailable, the app now explains the limited session rather than incorrectly asking the user to sign in again. A Retry Connection action is available from Discover and Add Service/Event.

The limited-session message clarifies that a listing is stored locally and is not yet submitted to the server. The app must not tell the user that a listing has been submitted for approval until the API confirms success.

The existing green/teal and cream visual identity, rounded controls and reusable screen banner have been retained to avoid an unnecessary redesign late in the assessment cycle.

4. Security, privacy and deployment considerations

Passwords are hashed using bcrypt and are not stored as plain text. The API issues signed JWTs and requires a production JWT_SECRET of at least 32 characters. Authentication routes are rate-limited. The API does not enable arbitrary browser origins by default.

Google SSO must be configured with the correct Firebase Android application, SHA fingerprints, google-services.json and server-side FIREBASE_PROJECT_ID. The server-side token verification must remain enabled. Do not enable ALLOW_UNVERIFIED_SSO in production.

The JSON-file database survives process restarts on a persistent disk, but data may be lost on hosts with ephemeral storage or redeploys. Use a managed database or a correctly configured persistent volume for production. Deploy the API over HTTPS.

The password-reset endpoint returns HTTP 501 because an email delivery workflow is not implemented. This limitation is disclosed rather than presented as a working feature.

5. Verification status and evidence

The project includes API tests, Android unit tests and a GitHub Actions workflow. However, automated execution could not be completed in the environment used to prepare this archive. npm install timed out before API dependencies were available. The Gradle wrapper could not download Gradle 8.11.1 because the environment could not resolve services.gradle.org. No automated test pass or successful APK build is claimed in this report.

Two owner-supplied photographs are included in docs/device-evidence. They show the app running on a physical Android phone on 4 November 2026: the Discover screen and the Add Service/Event form. The screenshots also show that the API was unavailable at the time. They demonstrate a physical-device UI state only; they do not prove successful API submission, SSO exchange, offline synchronisation or push delivery.

All manual test cases remain NOT RUN until the group performs them on the actual device and records the real outcome. The detailed register is docs/TEST_EVIDENCE.md. Replace NOT RUN only with genuine results, dates and evidence filenames.

6. Required final verification before submission

On a network-enabled development computer, run `cd server && npm install && npm test` and save the terminal output. In Android Studio with JDK 21 and the required Android SDK, run `./gradlew testDebugUnitTest` and `./gradlew assembleDebug`; save the test report and APK build evidence.

Configure and test the deployed API from the physical phone. Confirm /health returns HTTP 200. Test registration, login, invalid credentials, Google SSO, service search, event discovery, listing submission, pending approval, reviews, Room saved items, offline queueing, reconnect synchronisation, language changes and notification delivery.

Record a demonstration video showing the complete user journey. Add its real unlisted URL to README.md. Add real screenshots for successful online submission, offline queue and sync, settings/language, and notifications. Show a green GitHub Actions run only after the workflow succeeds.

Before upload, inspect the repository for secrets, tokens, private user data and generated database files. Confirm the README, report, test register and demonstrated app all describe the same implementation.

7. Known limitations

The current evidence package does not establish a successful live API connection from the physical phone. The API address and network/firewall/deployment configuration must be checked by the group.

Google SSO and push notification delivery depend on the group Firebase project and deployment configuration.

Automated tests and Android build are pending execution in a network-enabled environment. Physical-device end-to-end tests, the final demonstration video and successful Play Store publication evidence are also pending.

This portfolio is a truthful, improved submission package, not a guarantee of a particular mark. The final result is determined by the complete assessment rubric and lecturer verification.

Appendix A – Evidence files

• docs/TEST_EVIDENCE.md

• docs/FEATURE_EVIDENCE_CHECKLIST.md

• docs/POSTMAN_TEST_PLAN.md

• docs/device-evidence/discover-physical-device-api-unavailable.jpg

• docs/device-evidence/add-listing-physical-device-api-unavailable.jpg

Prepared from the supplied source archive and owner-provided physical-device photographs. No unperformed test is represented as passed.